agentic-ai-azure-week05-mcp-tools

Week 5 - Tools, MCP & Interoperability

Agentic AI on Azure - Enterprise Master Class | Handout (slides + notes)

Notes

Welcome. This deck covers 'agentic-ai-azure-week05-mcp-tools' - Week 5 - Tools, MCP & Interoperability. Frame the problem and why it matters, then walk the class through the learning goal, the enterprise use case, the architecture/flow, the key concepts, a live run in VS Code, and the architect's takeaways. The repo runs locally with no Azure, so demo it live.

Slide 2 - Learning goal

- Connect agents to tools via standardized contracts
- Build a reusable tool registry
- Make write tools idempotent

Notes

[Learning goal] Walk through these talking points:
- Connect agents to tools via standardized contracts
- Build a reusable tool registry
- Make write tools idempotent

Ask the class: which of these ideas is new to you?

Slide 3 - Enterprise use case

- Procurement Operations Agent
- Checks inventory, gets supplier pricing, creates a PO
- Each tool is a contract reusable across agents

Notes

[Enterprise use case] Walk through these talking points:
- Procurement Operations Agent
- Checks inventory, gets supplier pricing, creates a PO
- Each tool is a contract reusable across agents

Tip: relate this to a regulated scenario your students know.

Slide 4 - Architecture / flow

- Tool registry: check_inventory, get_supplier_pricing, create_purchase_order
- MCP-style discovery at GET /api/v1/tools
- Idempotency key -> retries return the same order
- Foundry tool-calling loop drives the tools

Notes

[Architecture / flow] Walk through these talking points:
- Tool registry: check_inventory, get_supplier_pricing, create_purchase_order
- MCP-style discovery at GET /api/v1/tools
- Idempotency key -> retries return the same order
- Foundry tool-calling loop drives the tools

Demo: trace a single request through these steps live.

Slide 5 - Key concepts

- JSON-schema tool contracts (MCP / function calling)
- Schemas as the trust boundary
- Gateway pattern: APIM for auth/limits/observability

Notes

[Key concepts] Walk through these talking points:
- JSON-schema tool contracts (MCP / function calling)
- Schemas as the trust boundary
- Gateway pattern: APIM for auth/limits/observability
Open the named file in the repo while you explain each point.

Slide 6 - Run it (VS Code - no Azure needed)

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/procure
- Runs in MOCK mode - no Azure/keys needed

Notes

[Run it (VS Code - no Azure needed)] Walk through these talking points:
- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/procure
- Runs in MOCK mode - no Azure/keys needed
Pause for questions before moving on.

Slide 7 - Architect's takeaways

- MCP decouples tools from agents (portability)
- Idempotency is essential for write tools
- Who can call what - govern the tool surface

Notes

[Architect's takeaways] Walk through these talking points:
- MCP decouples tools from agents (portability)
- Idempotency is essential for write tools
- Who can call what - govern the tool surface
Discussion: when would you NOT choose this approach?